



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Name: Operating System Lab

Course Code: CSE 402

LAB REPORT

Lab Report Number : 01

Lab Report Name : First Come First Serve (FCFS) Scheduling Algorithm.

Date of Submission : 02.08.2026

Submitted By:

Name : Ejaj Ahmmod Lemon

ID : 23101160

Section: C2

Submitted To:

Atia Rahman Orthi

Lecturer

University of Asia Pacific

1. Problem Statement:

The objective of this laboratory task is to design and implement the **First Come First Serve (FCFS) CPU Scheduling Algorithm** for managing process execution based on their arrival order. In FCFS scheduling, the process that arrives first is selected for execution before the processes that arrive later.

The implementation should handle processes with different arrival times and determine their execution sequence accordingly. After scheduling the processes, the program should calculate the essential scheduling parameters, including **Completion Time (CT)**, **Turnaround Time (TAT)**, and **Waiting Time (WT)** for each process. The results should then be presented clearly to evaluate the performance of the FCFS scheduling technique.

2. Objectives:

The main objectives of this laboratory experiment are:

- To understand the basic working principle of the **First Come First Serve (FCFS) CPU Scheduling Algorithm**.
- To implement FCFS scheduling by arranging processes according to their **arrival times**.
- To calculate the **Completion Time, Turnaround Time, and Waiting Time** for each process.
- To analyze the scheduling performance based on the calculated parameters.
- To examine the key **advantages and limitations** of the non-preemptive FCFS scheduling algorithm.
- To gain practical experience in implementing CPU scheduling algorithms using programming.

3. Source Code Screenshot:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Process {
5      string pid;
6      int arrival_time;
7      int burst_time;
8      int completion_time;
9      int turnaround_time;
10     int waiting_time;
11 };
12
13 bool compareByArrival(const Process &a, const Process &b) {
14     return a.arrival_time < b.arrival_time;
15 }
16
17 int main() {
18     int num_processes;
19     cout << "Enter the number of processes: ";
20     cin >> num_processes;
21
22     vector<Process> processes(num_processes);
23
24     cout << "Enter PID, Arrival Time, Burst Time for each process:\n";
25     for (int i = 0; i < num_processes; i++) {
26         cin >> processes[i].pid >> processes[i].arrival_time >> processes[i].burst_time;
27     }
28
29     sort(processes.begin(), processes.end(), compareByArrival);
30
31     int current_time = 0;
32     for (int i = 0; i < num_processes; i++) {
33         if (current_time < processes[i].arrival_time) {
34             current_time = processes[i].arrival_time;
35         }
36
37         current_time += processes[i].burst_time;
38         processes[i].completion_time = current_time;
39         processes[i].turnaround_time = processes[i].completion_time - processes[i].arrival_time;
40         processes[i].waiting_time = processes[i].turnaround_time - processes[i].burst_time;
41     }
42
43     cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
44     double total_waiting = 0, total_turnaround = 0;
45
46     for (const auto &p : processes) {
47         cout << p.pid << "\t" << p.arrival_time << "\t" << p.burst_time << "\t"
48             << p.completion_time << "\t" << p.turnaround_time << "\t" << p.waiting_time << endl;
49         total_waiting += p.waiting_time;
50         total_turnaround += p.turnaround_time;
51     }
52
53     cout << fixed << setprecision(2);
54     cout << "\nAverage Waiting Time: " << total_waiting / num_processes << endl;
55     cout << "Average Turnaround Time: " << total_turnaround / num_processes << endl;
56
57     return 0;
58 }
```

Figure 1: Source Code of the Optimized FCFS Scheduling Algorithm

4. Output Screenshot:

```
Enter the number of processes: 5
Enter PID, Arrival Time, Burst Time for each process:
p0 3 1
p1 5 3
p2 2 2
p3 1 2
p4 6 3

PID      AT      BT      CT      TAT      WT
p3        1        2        3        2        0
p2        2        2        5        3        1
p0        3        1        6        3        2
p1        5        3        9        4        1
p4        6        3       12        6        3

Average Waiting Time: 1.40
Average Turnaround Time: 3.60
```

Figure 2: Execution Output of the FCFS Scheduling Algorithm

5. Discussion:

The implemented **First Come First Serve (FCFS) scheduling algorithm** provides an efficient approach to process scheduling by arranging processes according to their arrival times before execution. This ensures that processes are handled in the correct order without repeatedly searching for the next process during execution.

The implementation first collects the required information for all processes and then sorts them based on their arrival time using a custom comparator. A **current time counter** is maintained throughout the scheduling process to determine when each process can begin execution. If the CPU remains idle because the next process has not yet arrived, the current time is advanced to that process's arrival time.

Several important observations were obtained from the implementation:

- The sorting operation results in an overall **time complexity of $O(n \log n)$** , which is more efficient than the **$O(n^2)$** approach used in some naive implementations.
- CPU idle periods are handled efficiently by advancing the current time directly to the arrival time of the next available process.
- **Waiting Time (WT)** can be calculated using the relationship between Turnaround Time and Burst Time.

- Since FCFS is a **non-preemptive scheduling algorithm**, once a process begins execution, it continues until its completion.
- FCFS provides a simple and predictable scheduling mechanism; however, it may result in a higher **average waiting time** when a long-duration process arrives before shorter processes.
- The implementation demonstrates that although FCFS is simple and easy to implement, its performance may not be optimal for workloads where minimizing waiting time is a major concern.

Overall, the experiment provided practical understanding of the **FCFS CPU scheduling mechanism**, process ordering, scheduling metrics, CPU idle-time handling, and the advantages and limitations of non-preemptive scheduling.

6. Conclusion:

The implementation of the **First Come First Serve (FCFS) CPU Scheduling Algorithm** successfully demonstrated the fundamental concepts of process scheduling and execution based on arrival time. The experiment also provided practical experience in calculating important scheduling parameters such as **Completion Time, Turnaround Time, and Waiting Time**.

Although FCFS is simple, fair, and easy to implement, it has certain limitations, particularly the **convoy effect**, where shorter processes may have to wait for a longer process to complete. As a result, FCFS may not always provide optimal average waiting and turnaround times.

The use of sorting based on arrival time improves the efficiency of the implementation while preserving the fundamental behavior of FCFS scheduling. Overall, the experiment enhanced the understanding of **non-preemptive CPU scheduling**, algorithmic efficiency, and the practical advantages and limitations of the FCFS scheduling approach.